

HOCHSCHULE BREMERHAVEN

FACHBEREICH 02

MANAGEMENT UND INFORMATIONSSYSTEME

Parallele Algorithmen Multicore-Praktikum

Assignments

PRÜFER:IN: Prof. Dr. Thomas Umland

SEMESTER: Sommersemester 2025

NAME: Hossein Akbari

MATR.-NR.: 39940

E-MAIL: hosakbari@smail.hs-bremerhaven.de

FACHSEMESTER: 06

STUDIENGANG: B.Sc. Informatik

Abgabedatum: 16. Juni 2025

Inhaltsverzeichnis

1	Einführung	1
2	Hauptteil	2
2.1	Odd Even Merge Splitting Sort	2
2.1.1	Übungen zum Odd Even Merge Splitting Sort	4
2.1.2	Assignment 1	9
2.2	Bitonic Sort	24
2.2.1	Übungen zum Bitonic Sort	25
2.2.2	Assignment 2	31
2.3	Merge Sort	38
2.3.1	Übungen zum parallelen Merge Sort	38
2.3.2	Assignment 3	38
2.4	Amdahls Gesetz und Gustafsons Interpretation	38
3	Schluss	39
3.1	Reflexion	39

3.2	Fazit	39
-----	-----------------	----

1 Einführung

Unter der Anweisung von Prof. Dr. Thomas Umland sollen für einige Sortieralgorithmen, die in der Parallelverarbeitung zum Einsatz kommen, auf dem Multicore-Rechner, für eine gegebene Problemgröße N Laufzeitmessungen durchgeführt werden. Dabei wird jede Messung dreimal wiederholt und die kürzeste Laufzeit als Ergebnis herangezogen. Anschließend werden Laufzeit, relative Beschleunigung und Effizienz visualisiert und sowohl mit den theoretischen Vorüberlegungen als auch mit den Messergebnissen anderer Algorithmen verglichen.

Im Mittelpunkt des Moduls steht das Ziel, die Ausführungszeit großer Sortierprobleme durch Parallelisierung auf mehreren Kernen zu verkürzen. Dazu werden verschiedene parallele Merge-Split-Algorithmen analysiert, implementiert und auf einem Multicore-Rechner ausgeführt. Dadurch soll praktische Erfahrung mit der Parallelprogrammierung gesammelt, sowie der Umgang mit Race Conditions und Synchronisationsmechanismen erlernt werden. Ferner gibt es dabei eine intensive Auseinandersetzung mit den Kennzahlen „Beschleunigung“ und „Effizienz“.

Die Lernziele dieses Multicore-Praktikums umfassen zum einen das Verständnis, wie Beschleunigung und Effizienz definiert und gemessen werden. Zum anderen sollen die Teilnehmenden typische Ursachen für Abweichungen vom idealen Skalierungsverhalten – wie Synchronisations- und Kommunikations overhead oder Lastverteilungsschwierigkeiten – erkennen und deren Einfluss durch eigene Messungen quantifizieren.

Relative Beschleunigung

Die *relative Beschleunigung* $r_n(p)$ eines parallelen Algorithmus ist definiert als

$$r_n(p) := \frac{t_n(1)}{t_n(p)},$$

wobei $t_n(1)$ die Laufzeit auf einem einzelnen Prozessor und $t_n(p)$ die Laufzeit auf p Prozessoren angibt.

Effizienz

Die *Effizienz* $e_n(p)$ ergibt sich aus der relativen Beschleunigung dividiert durch die Prozessorzahl:

$$e_n(p) := \frac{r_n(p)}{p}.$$

Im weiteren Verlauf dieser Arbeit werden die in den Assignments erhobenen Messwerten diskutiert und mit den theoretischen Vorhersagen abgeglichen.

2 Hauptteil

2.1 Odd Even Merge Splitting Sort

Odd–Even Transposition Sort Der Odd–Even Transposition Sort ist ein einfacher, prozessorientierter Vergleichssortieralgorithmus. Für n zu sortierende Elemente werden n Prozessoren $P_1, \dots, P_n$ verwendet, wobei Prozessor P_i genau ein Element z_i speichert. Der Algorithmus besteht aus n Runden, in denen abwechselnd ungerade und gerade Paare verglichen und getauscht werden:

1. *Ungerader Schritt:* Alle Prozessoren mit ungeradem Index i , $1 \leq i < n$, vergleichen simultan ihre Elemente z_i und z_{i+1} . Sind sie in falscher Reihenfolge ($z_i > z_{i+1}$), so tauschen die beiden Prozessoren ihre Werte.
2. *Gerader Schritt:* Alle Prozessoren mit geradem Index j , $1 < j \leq n - 1$, führen denselben Vergleichs- und Tauschschritt mit ihrem Nachbarn durch.

Nach n solchen Phasen liegen die Elemente in aufsteigender Reihenfolge in $P_1, \dots, P_n$.

Bemerkungen

- Der Algorithmus ist hardwareeffizient, da alle Vergleiche lokal und synchronisierbar ablaufen.
- In einer Softwareimplementierung auf weniger als n Kernen entsteht jedoch ein starkes Missverhältnis zwischen Rechenaufwand und Kommunikation, wenn jeder Kern nur wenige Elemente bearbeitet.

Odd–Even Merge–Splitting Sort Um die Last auf einem System mit $p \leq n$ Prozessoren besser auszunutzen, verallgemeinert der Odd–Even Merge–Splitting Sort das oben beschriebene Verfahren:

1. **Initialisierung:** Die n Eingabeelemente werden gleichmäßig auf die p Kerne verteilt; jeder Kern hält ein lokales Array A_i der Länge n/p .
2. **Lokales Sortieren:** Jeder Kern sortiert sein Teilarray A_i sequenziell (z. B. mit Heapsort oder Bubble Sort).
3. **Merge–Split–Phase:** Es folgen p Runden, in denen abwechselnd *odd* und *even* Kerne aktiv sind:
 - In einer *odd*-Runde führt jeder ungerade Kern i simultan mit Kernel $i + 1$ eine Merge–Split-Operation auf (A_i, A_{i+1}) durch:
 - *Merge*: Erzeuge aus den beiden sortierten Listen A_i und A_{i+1} eine zusammenhängende sortierte Sequenz der Länge $2 \cdot (n/p)$.
 - *Split*: Weise den unteren Teil (erste n/p Elemente) zurück an A_i und den oberen Teil an A_{i+1} .
 - In der folgenden *even*-Runde wiederholen alle geraden Kerne i denselben Merge–Split-Schritt mit Kern $i + 1$.
4. **Fertigstellung:** Nach p Runden stehen alle n Elemente global sortiert über die Arrays $A_1, \dots, A_p$ verteilt.

Pseudocode

Algorithm 1 Odd–Even Merge–Splitting Sort

```
1: procedure OEMERGESPLITSORT( $A[0..n-1], p$ )
2:   Aufteilen:  $\{A_i\}_{i=1}^p \leftarrow$  Blöcke der Länge  $n/p$ 
3:   for all Kern  $i = 1, \dots, p$  parallel do
4:     Sortiere  $A_i$  sequenziell
5:   end for
6:   for  $k \leftarrow 1$  to  $p$  do
7:     if  $k$  ungerade then
8:       for all ungerade  $i = 1, 3, \dots, p-1$  parallel do
9:         Merge–Split( $A_i, A_{i+1}$ )
10:      end for
11:     else
12:       for all gerade  $i = 2, 4, \dots, p-1$  parallel do
13:         Merge–Split( $A_i, A_{i+1}$ )
14:       end for
15:     end if
16:   end for
17:   return Zusammengeführtes Array
18: end procedure
```

2.1.1 Übungen zum Odd Even Merge Splitting Sort

Welche Laufzeit (in O-Notation) besitzt der Odd Even Merge Splitting-Algorithmus, wenn zum Sortieren der lokalen Listen der sequentielle Heap Sort-Algorithmus verwendet wird?

Der Algorithmus zerfällt in zwei Phasen:

1. Lokale Sortierung:

Jeder der p Prozesse sortiert seine $\frac{n}{p}$ Elemente mit Heapsort. Da Heapsort in $O(m \log m)$ arbeitet, ergibt sich für jeden Prozess

$$f = O\left(\frac{n}{p} \log \frac{n}{p}\right).$$

2. Merge-Split-Phase:

Es folgen p Runden des Odd-Even-Splittings. In jeder Runde führen benachbarte Prozesse einen Merge ihrer beiden Teillisten (jeweils $\frac{n}{p}$ Elemente) in $O(\frac{n}{p})$ durch und splitten das Ergebnis ebenfalls in $O(\frac{n}{p})$. Damit beträgt der Aufwand pro Runde

$$O\left(\frac{n}{p}\right) + O\left(\frac{n}{p}\right) = O\left(\frac{n}{p}\right),$$

und über alle p Runden

$$g = p \cdot O\left(\frac{n}{p}\right) = O(n).$$

Gesamtlaufzeit: Mit $f = O\left(\frac{n}{p} \log\left(\frac{n}{p}\right)\right)$ und $g = O(n)$ gilt allgemein

$$t_n(p) = O(f + g) = O\left(\frac{n}{p} \log\left(\frac{n}{p}\right)\right) + O(n) = O\left(\max\left\{\frac{n}{p} \log\left(\frac{n}{p}\right), n\right\}\right).$$

Umformung und Fallunterscheidung Zunächst wendet man die Logarithmusregel $\log(a/b) = \log a - \log b$ an:

$$\frac{n}{p} \log\left(\frac{n}{p}\right) = \frac{n}{p} (\log n - \log p) = \frac{n}{p} \log n - \frac{n}{p} \log p = O\left(\frac{n}{p} \log n\right).$$

Vergleicht man nun $\frac{n}{p} \log n$ mit n , so erhält man

$$\frac{n}{p} \log n \geq n \iff \frac{\log n}{p} \geq 1 \iff \log n \geq p.$$

Daraus ergibt sich die Fallunterscheidung:

$$t_n(p) = \begin{cases} O\left(\frac{n}{p} \log n\right), & p \leq \log n, \\ O(n), & p > \log n. \end{cases}$$

Welcher Verlauf ergibt sich in diesem Fall für die Beschleunigung?

Die relative Beschleunigung $r_n(p)$ ist definiert als

$$r_n(p) = \frac{t_n(1)}{t_n(p)},$$

wobei $t_n(1)$ die Laufzeit auf einem Prozessor und $t_n(p)$ die Laufzeit auf p Prozessoren ist.

Laufzeiten einsetzen Für einen Prozessor gilt

$$t_n(1) = O(n \log n) + O(n) = O(n \log n).$$

Für p Prozessoren hatten wir bereits

$$t_n(p) = \begin{cases} O\left(\frac{n}{p} \log n\right), & p \leq \log n, \\ O(n), & p > \log n. \end{cases}$$

Stückweise Beschleunigung Setzt man dies in $r_n(p) = t_n(1)/t_n(p)$ ein, so erhält man

$$r_n(p) = \frac{O(n \log n)}{O(\max\{\frac{n}{p} \log n, n\})} = O\left(\min\left\{\frac{n \log n}{(n/p) \log n}, \frac{n \log n}{n}\right\}\right) = \begin{cases} O(p), & p \leq \log n, \\ O(\log n), & p > \log n. \end{cases}$$

Interpretation

- Für $p \leq \log n$ skaliert der Algorithmus nahezu linear: Jede Verdopplung von p verdoppelt annähernd die Beschleunigung.
- Ab $p > \log n$ erreicht man eine logarithmische Sättigung: Zusätzliche Prozessoren bringen nur noch eine Beschleunigung in $O(\log n)$.

Wie groß ist die Effizienz?

Die Effizienz $e_n(p)$ ist definiert als

$$e_n(p) = \frac{r_n(p)}{p},$$

wobei $r_n(p)$ die hergeleitete relative Beschleunigung ist:

$$r_n(p) = \begin{cases} O(p), & p \leq \log n, \\ O(\log n), & p > \log n. \end{cases}$$

Setzt man dies in $e_n(p) = r_n(p)/p$ ein, so erhält man

$$e_n(p) = \begin{cases} O\left(\frac{p}{p}\right) = O(1), & p \leq \log n, \\ O\left(\frac{\log n}{p}\right), & p > \log n. \end{cases}$$

Interpretation

- Für $p \leq \log n$ ist $e_n(p) = O(1)$: Die Prozessoren werden nahezu vollständig ausgelastet, d. h. eine Verdopplung von p liefert nahezu doppelte Beschleunigung pro Kern.
- Für $p > \log n$ fällt $e_n(p) = O\left(\frac{\log n}{p}\right)$ mit wachsendem p : Zusätzliche Kerne bringen immer weniger Speedup pro Kopf, bis ab einem gewissen Punkt praktisch kein Gewinn mehr erzielt wird.

Wie ändern sich die Ausdrücke für die Beschleunigung und die Effizienz, wenn zum Sortieren der lokalen Listen der sequentielle Bubble Sort-Algorithmus verwendet wird?

Verwendet man zum lokalen Sortieren den sequentiellen Bubble-Sort-Algorithmus, so ändern sich die Kosten folgendermaßen:

1. Gesamtlaufzeit

- (a) **Lokale Sortierung:** Jeder der p Prozesse sortiert seine $\frac{n}{p}$ Elemente mit Bubble Sort in

$$f = O\left(\left(\frac{n}{p}\right)^2\right).$$

- (b) **Merge-Split-Phase:** Wie bisher werden p Runden ausgeführt, jeweils mit Aufwand $O(\frac{n}{p})$ pro Prozess, insgesamt

$$g = p \cdot O\left(\frac{n}{p}\right) = O(n).$$

- (c) **Gesamtaufwand:**

$$t_n(p) = O\left(\left(\frac{n}{p}\right)^2\right) + O(n) = O(\max\{\frac{n^2}{p^2}, n\}).$$

Damit ergibt sich

$$t_n(p) = \begin{cases} O(\frac{n^2}{p^2}), & p \leq \sqrt{n}, \\ O(n), & p > \sqrt{n}. \end{cases}$$

2. Relative Beschleunigung Die Beschleunigung ist

$$r_n(p) = \frac{t_n(1)}{t_n(p)}, \quad t_n(1) = O(n^2) + O(n) = O(n^2).$$

Da $t_n(p) = O(\max\{n^2/p^2, n\})$, folgt

$$r_n(p) = O\left(\min\left\{\frac{n^2}{(n^2/p^2)}, \frac{n^2}{n}\right\}\right) = O(\min\{p^2, n\}),$$

also

$$r_n(p) = \begin{cases} O(p^2), & p \leq \sqrt{n}, \\ O(n), & p > \sqrt{n}. \end{cases}$$

3. Effizienz Die Effizienz $e_n(p) = r_n(p)/p$ ergibt

$$e_n(p) = \begin{cases} O\left(\frac{p^2}{p}\right) = O(p), & p \leq \sqrt{n}, \\ O\left(\frac{n}{p}\right), & p > \sqrt{n}. \end{cases}$$

Interpretation

- Für $p \leq \sqrt{n}$ steigt die Effizienz $O(p)$ stark mit der Prozessorzahl, da Bubble Sort besonders teuer ist und jeder neue Kern einen großen Anteil an der Arbeit übernimmt.
- Ab $p > \sqrt{n}$ fällt $e_n(p) = O\left(\frac{n}{p}\right)$ mit wachsendem p : Zusätzliche Kerne tragen immer weniger zur Beschleunigung bei, bis ein weiterer Gewinn praktisch verschwindet.

2.1.2 Assignment 1

In Assignment 1 sollen mit dem bereits implementierten Odd-Even-Merge-Splitting-Algorithmus in Java mit zwei Varianten für den sequentiellen Sortierschritt:

- **Variante 1:** Lokales Sorting mit `Arrays.sort()`, sowie
- **Variante 2:** Lokales Sorting mit Bubble Sort,

Laufzeitmessungen auf dem Multicore-Rechner durchgeführt werden. Die Messung soll für verschiedene Problemgrößen n und unterschiedliche Thread- bzw. Prozessoranzahlen p erfolgen. Weiterhin sollen die Laufzeit, basierend daraus auch Beschleunigung und Effizienz berechnet und visualisiert werden. Abschließend sollen die gemessenen Werte mit den theoretischen Überlegungen aus den vorherigen Aufgaben verglichen und beurteilt werden, welche Variante in der Praxis besser abschneidet.

Durchgeführte Experimente

In Aufgabe 01 wurde mit `Arrays.sort()` und den Problemgrößen $n = 921\,600\,000$ und $n = 1\,843\,200\,000$, Prozessorzahlen $p = 1, \dots, 24$ gemessen. In Aufgabe 02 wurde mit Bubble Sort und den Problemgrößen $n = 500\,000$, Prozessorzahlen (Threads) $p = 1, \dots, 140$ gemessen. Aus den gemessenen Laufzeiten wurden Beschleunigung und Effizienz für jedes p berechnet und grafisch dargestellt.

Aufgabe 01 (`Arrays.sort()`)

Problemgrößen $n = 921\,600\,000$ und $n = 1\,843\,200\,000$, Prozessorzahlen $p = 1, \dots, 24$

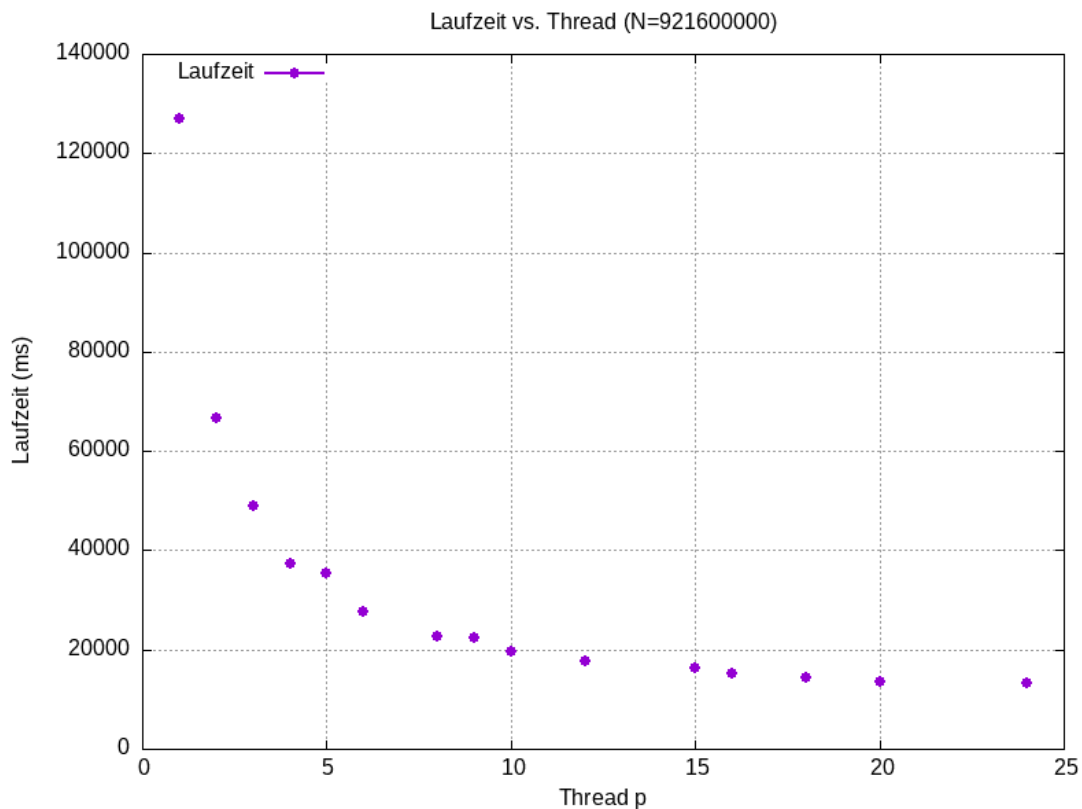


Abbildung 1: Laufzeit vs. Thread (N=921 600 000)

Der Algorithmus braucht bei einem Thread am meisten. Wenn jedoch mehr Thread eingesetzt werden, wird die Laufzeit weniger. Also je mehr Threads, desto weniger Laufzeit. Bei zwei Threads halbiert sich die Laufzeit fast. Bei vier Threads ist es schon ca. bei 48 Sekunden. Mit acht ist es bei ca. 22 Sekunden. Bis zwölf Threads fällt die Laufzeit sehr steil ab. Ab 12 Threads, wird aber der Gewinn pro Thread weniger. Grund hierfür ist eventuell, weil die Synchronisations- und Merge-Phasen mitwachsen.

Theoretische Überlegung aus Aufgabe 3.3.2: Hier wird gesagt, dass solange p kleiner als $\log n$ der lokale Heap-Sort dominiert. Das führt zu annähernd umgekehrter Proportionalität. Bei unserem Diagramm kann man genau das gut beobachten: Bis 12 Threads ist der Abfall steil, ab 12 Threads gibt es eine leichte Abflachung durch Parallel-Overhead.

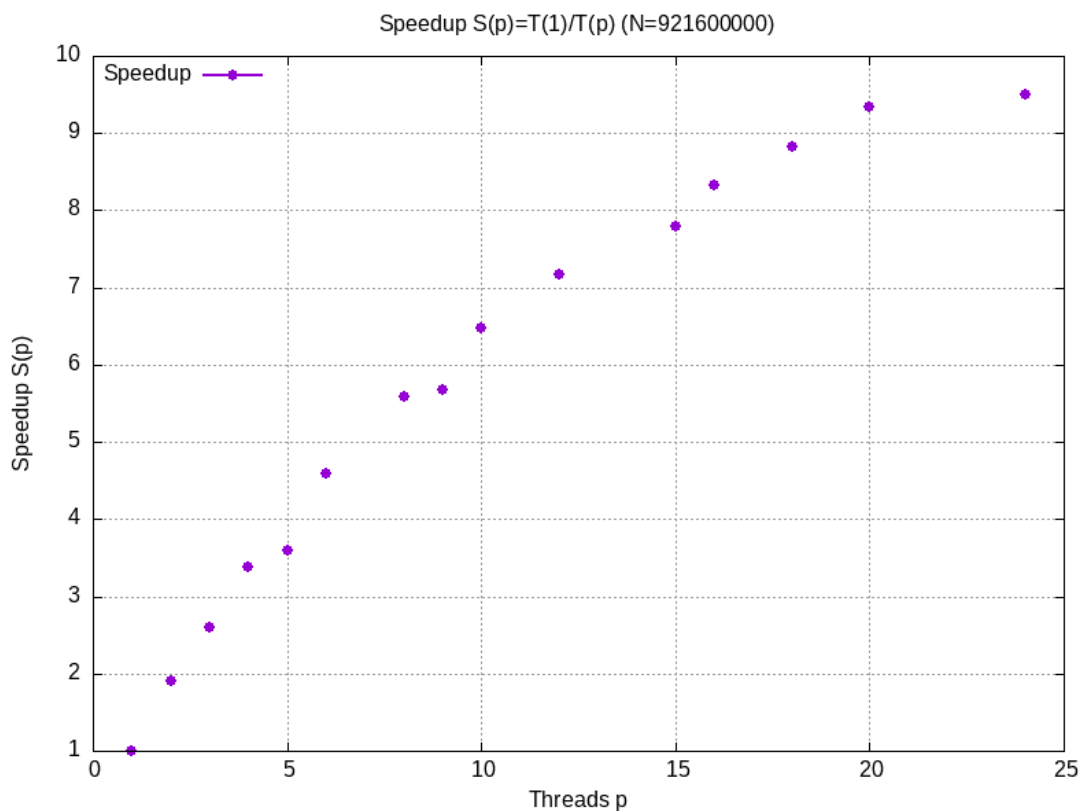


Abbildung 2: Speedup $S(p) = T(1)/T(p)$ ($N=921600000$)

Die Beschleunigung startet bei 1. Mit zwei Threads ist sie fast äquivalent zu 2. Bei vier sind es ca. bei 3,4. Mit acht sind es ca. bei 6,5. Und mit 24 Threads knapp bei 9,5. Diese Beobachtung ist nahezu ideal, also Beschleunigung als Äquivalenz zu P . Aber eigentlich zieht es etwas nach unten hin, weil das Zusammenfügen der Teillisten Arbeit kostet.

Theoretische Überlegung aus Aufgabe 3.3.2: Hier wird gesagt, dass für $p \leq \log n$ die Beschleunigung linear ist: $S(p) = O(p)$. Da wir bis $p = 24 < 30$ messen, sollte $S(p)$ fast genau proportional mitwachsen zu p . Das kann man anhand der Kurve gut sehen.

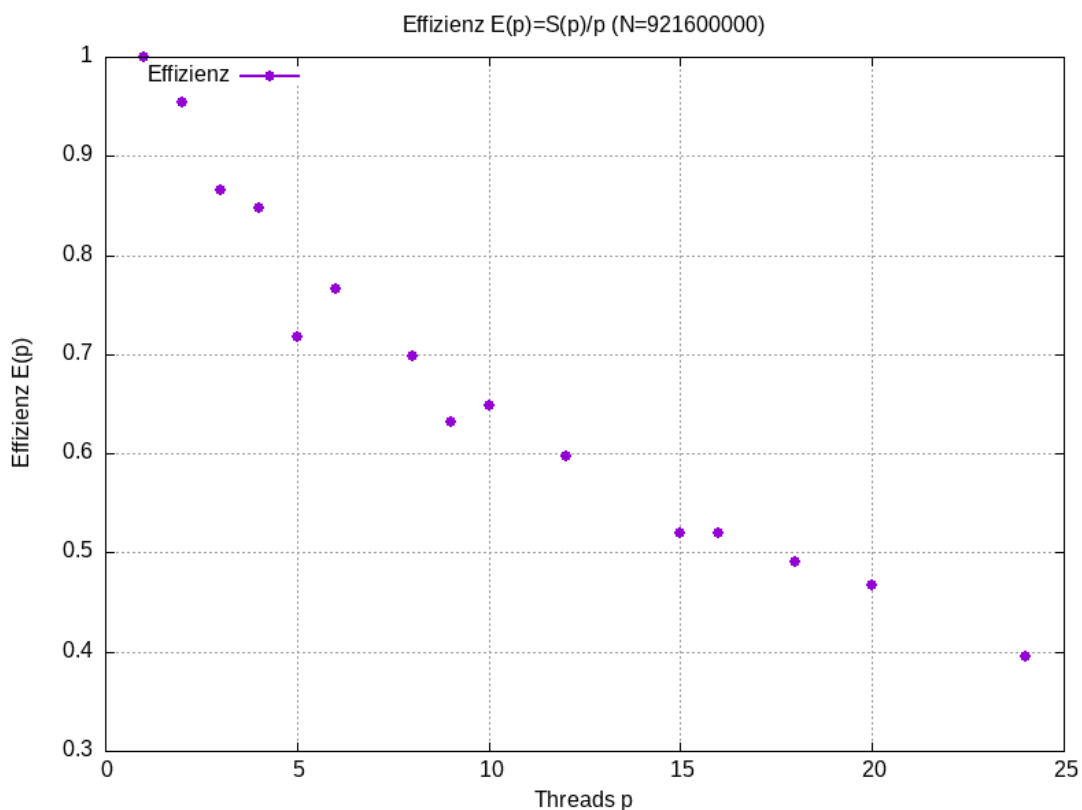


Abbildung 3: Effizienz $E(p)=S(p)/p$ ($N=921600000$)

Kurze Randinformation: Bei Effizienz wird gemessen, wieviel Beschleunigung man pro Kern rausholt. Bei $P = 1$, ist die Effizienz gleich 1. Wenn die Kerne mehr werden, fällt die Effizienz kontinuierlich auf Werte um ca. 0,85 (bei 4 Kernen), 0,63 (8 Kernen), 0,52 (16 Kernen), bis knapp 0,40 (24 Kernen). Jeder weitere Kern bringt bisschen weniger relative Beschleunigung. Die Kurve fällt zwar langsam, aber stetig.

Theoretische Überlegung aus Aufgabe 3.3.2: Hier wird prognostiziert, dass für $P \leq \log$ eine konstante Effizienz $E(p) = O(1)$ gilt. Sprich: Es gibt nur einen langsamen Abfall durch Overhead. Genau das kann man sehen. $E(p)$ bleibt im Bereich 0,4 - 1,0 und sinkt nur mäßig, weil die Merge- und Synchronisationskosten allmählich ins Gewicht fallen.

Gesamtfazit:

Die Laufzeit fällt annähernd linear mit $1/p$, solange p deutlich unter $\log n$ bleibt. Die Beschleunigung wächst fast proportional zu p , also die Parallelisierung skaliert hervorragend. Die Effizienz bleibt im Bereich ca. 0,4 bis 1,0 und sinkt nur langsam, weil man Overhead schwer vermeiden kann.

Insgesamt entsprechen die gemessenen Ergebnisse sehr gut Ergebnissen aus der Theorie.

Laufzeit, Beschleunigung und Effizienz als Tabelle für $N = 921\,600\,000$ (Nachtrag)

Tabelle 1: Laufzeit, Speed-up und Effizienz für $N = 921,600,000$

p	Laufzeit [ms]		Speed-up		Effizienz	
	Wert	%	Wert	%	Wert	%
1	127082	—	1.00	—	1.00	—
2	66627	-47.5	1.91	+90.8	0.95	-5.0
3	48945	-26.6	2.60	+36.0	0.87	-8.9
4	37471	-23.5	3.39	+30.7	0.85	-2.0
5	35397	-5.5	3.59	+5.9	0.72	-15.3
6	27639	-21.9	4.60	+28.0	0.77	+6.7
8	22752	-17.7	5.59	+21.5	0.70	-8.9
9	22365	-1.7	5.68	+1.7	0.63	-9.6
10	19596	-12.4	6.49	+14.1	0.65	+2.9
12	17736	-9.5	7.17	+10.5	0.60	-8.0
15	16303	-8.1	7.80	+8.7	0.52	-12.9
16	15266	-6.3	8.32	+6.8	0.52	0.0
18	14390	-5.8	8.83	+6.1	0.49	-5.6
20	13609	-5.4	9.34	+5.8	0.47	-4.9
24	13378	-1.7	9.50	+1.7	0.40	-15.2

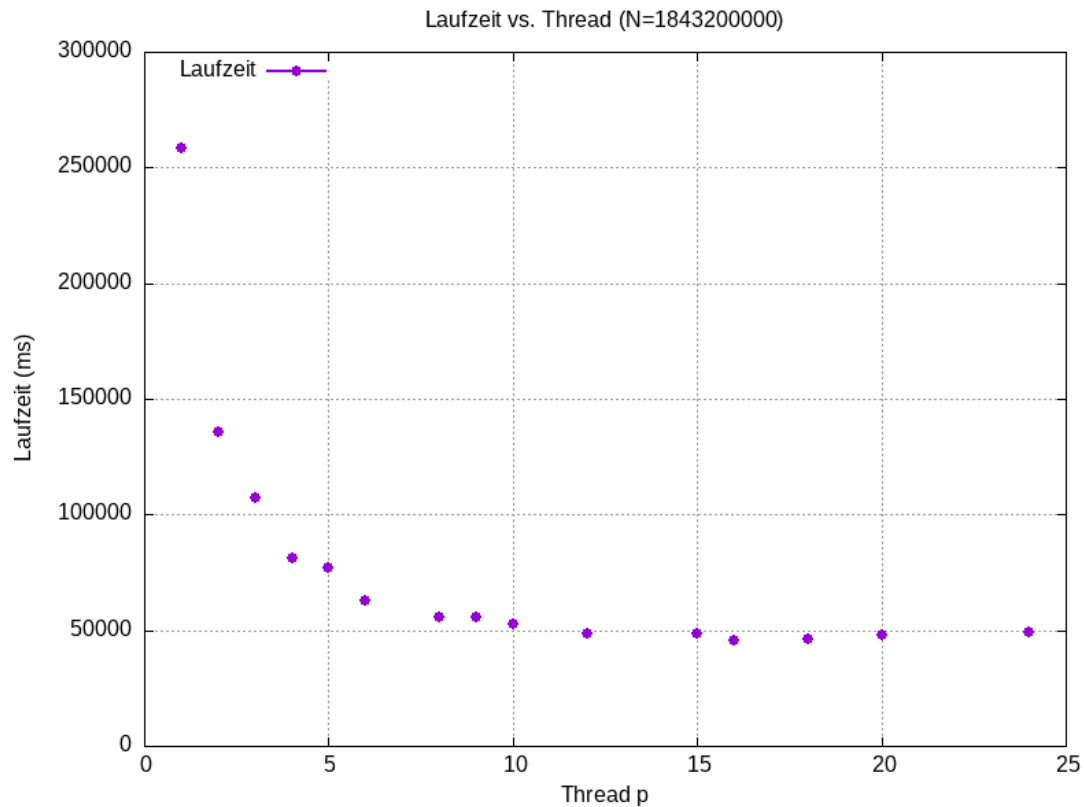


Abbildung 4: Laufzeit vs. Thread (N=1843200000)

Mit einem Kern ist die Laufzeit ca. 260000 ms. Bei 2 Kernen halbiert sich die Zeit auf ca. 135000 ms. Bei 4 Kernen ca. 108000 ms. Bei 6 Kernen ca. 80000 ms. Bei 8 Kernen => 76000 ms. 12 Kerne sind ca. 55000 ms. Ab 12 Kernen wird die Kurve deutlich flacher, was heißt, dass der Gewinn pro mehr Kern schrumpft. Beispiel hierfür ist zwischen 16 - 24 Kernen. Hier ändert sich die Zeit nur von 47000 auf 50000.

Theoretische Überlegung aus Aufgabe 3.3.2: Für $p \leq \log_2 n$ (ca.=) 31 erwartet man dass die Laufzeit ca. äquivalent ist zu $t(1)/p$. => Überwiegend $O(n/p \cdot \log n)$. Danach dominiert die feste Merge-Phase $O(n)$ und der Zeitgewinn durch zusätzliche Kerne verschwindet. Also: Die gemessene Laufzeit bei unserem Diagramm fällt nahezu im $1/p$ Verhältnis. Aber ab 12 Kernen sind die Merge- und Synchronisationen stärker.

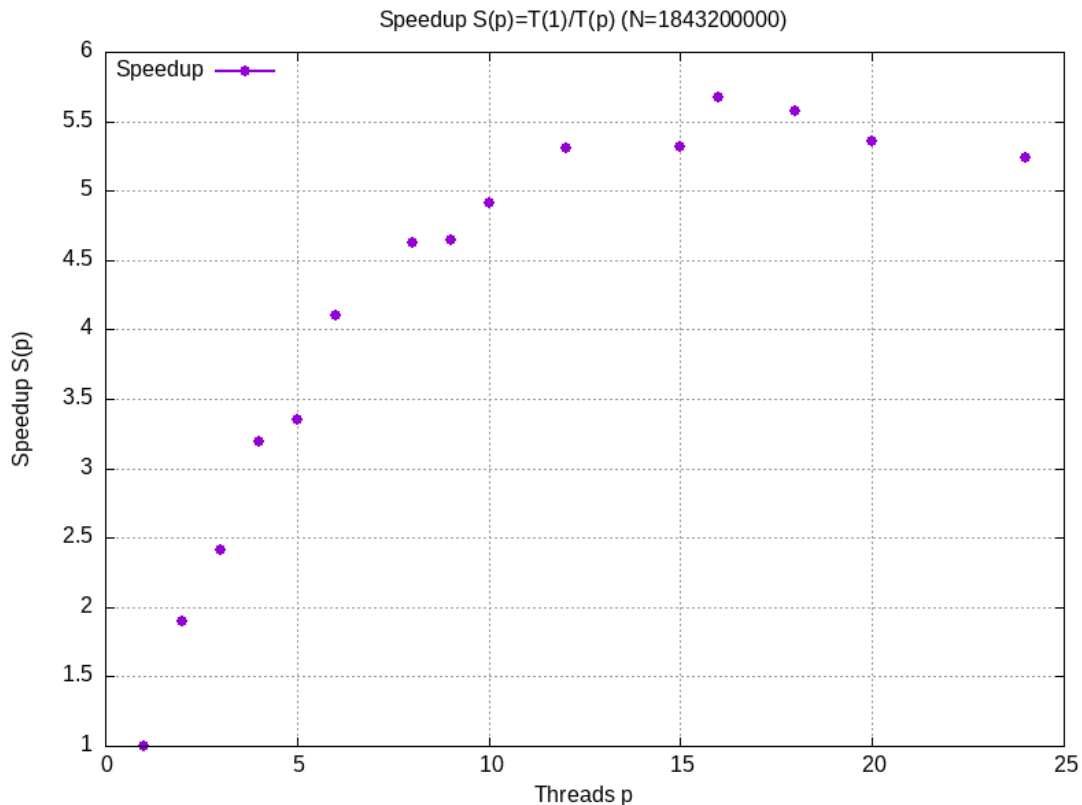


Abbildung 5: Speedup $S(p)=T(1)/T(p)$ ($N=1843200000$)

Die Beschleunigung ist bei 1 äquivalent zu 1. Also mit einem Kern ist die Beschleunigung 1. Bei zwei Kernen ca. 1,9. Bei 4 => 3,2. Bei 6 => 4,1. Bei 8 = 4,3. 12 sind 4,8. Die höchste Beschleunigung haben wir bei 16 Kernen mit 5,6. Ab 18-24 Kernen sind die Beschleunigung leicht auf 5,2.

Theoretische Überlegung aus Aufgabe 3.3.2: Solange $P \leq \log_2 n$ ca. 31, erwartet man eine lineare Skalierung, wie oben bereits erwähnt also $S(p) = O(p)$. In diesem Bereich sollte die Beschleunigung mit jedem Kern ca. um Faktor 1 wachsen. Aus der Beobachtung sehen wir bis ca. 12-16 Kernen tatsächlich fast eine proportionale Steigerung. Danach drückt der feste Overhead der Merge- und Synchronisationsphasen, wodurch zusätzliche Kerne dann kaum noch echten Mehrgeinn für Beschleunigung bringen. Zuzolge fällt die Beschleunigung leicht ab.

Die Beschleunigung folgt also erstmal ganz gut einen linearen Verlauf, aber vir $p = \log_2 n$ ab 16 Kernen flacht sie ab, weil Merge- und Barriere-Synchronisation den Zuwachs von Parallelität wieder auffressen.

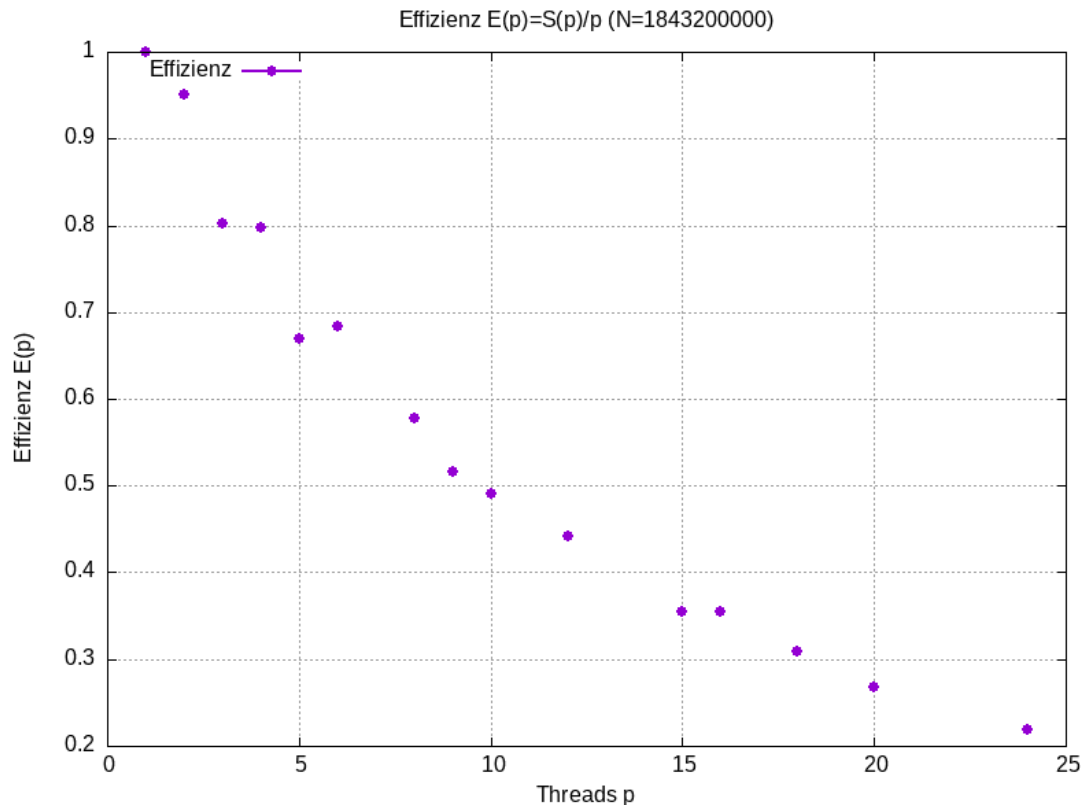


Abbildung 6: Effizienz $E(p)=S(p)/p$ ($N=1843200000$)

Mit einem Kern ist die Effizienz definitiv 100%. Bei zwei Kernen ca. 0,95. Bei 4 Kernen ca. 0,80. Bei 6 Kernen auch ca. 0,80. Bei 8 Kernen 0,67. Bei 12 Kernen ist es 0,58. Bei 16 Kernen 0,51. Bei 18 Kernen 0,31. 20 Kernen machen 0,27 aus und 24 gleich 0,22.

Theoretische Überlegung aus Aufgabe 3.3.2: Für $p \leq \log 2$ sollte die Effizienz $E(p)$ gleich $O(1)$ bleiben, also in etwa konstant, weil die Overhead-Kosten pro Kern vergleichsweise klein sind. Ab 6 Kernen fällt Effizienz deutlich unter 0,6, bis knapp- 0,2 bei 24 Kernen. Der Abfall ist hierbei stärker ausgeprägt als bei kleineren Problemgrößen, weil die feste Merge-Synchronisationsphase $O(n)$ im Verhältnis zur stark verkleinerten lokalen Arbeit $O(n/p \log n)$ schwerer ins Gewicht fällt.

Hierzu lässt sich sagen, dass die Effizienz mit jedem Kern sichtbar zurückgeht. Grund hierzu ist, dass bei großen n und vielen Threads der nicht-parallele Merge-Overhead und JVM-Management-Kosten dominieren. Das entspricht grob der theoretischen Erwartungen.

Laufzeit, Beschleunigung und Effizienz als Tabelle für $N = 1\,843\,200\,000$ (Nachtrag)

Tabelle 2: Laufzeit, Speed-up und Effizienz für $N = 1\,843\,200\,000$

p	Laufzeit [ms]		Speed-up		Effizienz	
	Wert	%	Wert	%	Wert	%
1	258907	—	1.00	—	1.00	—
2	135966	−47.4	1.90	+90.2	0.95	−4.9
3	107341	−21.0	2.41	+26.7	0.80	−15.6
4	81071	−24.4	3.19	+32.4	0.80	−0.7
5	77268	−4.7	3.35	+4.9	0.67	−16.1
6	63032	−18.4	4.10	+22.6	0.68	+2.2
8	55927	−11.3	4.63	+12.7	0.58	−15.5
9	55653	−0.5	4.65	+0.4	0.52	−10.7
10	52602	−5.9	4.92	+5.8	0.49	−4.7
12	48717	−7.4	5.31	+8.0	0.44	−10.1
15	48666	−0.1	5.31	+0.1	0.35	−19.9
16	45582	−6.3	5.23	−1.6	0.33	−7.7
18	46404	+1.8	5.58	+6.6	0.31	−5.2
20	48266	+4.0	5.35	−4.0	0.27	−13.5
24	49361	+2.3	5.24	−2.1	0.22	−18.7

Aufgabe 02 (Bubble-Sort)

Problemgröße $n = 500\,000$, **Prozessorzahlen (Threads)** $p = 1, \dots, 140$.

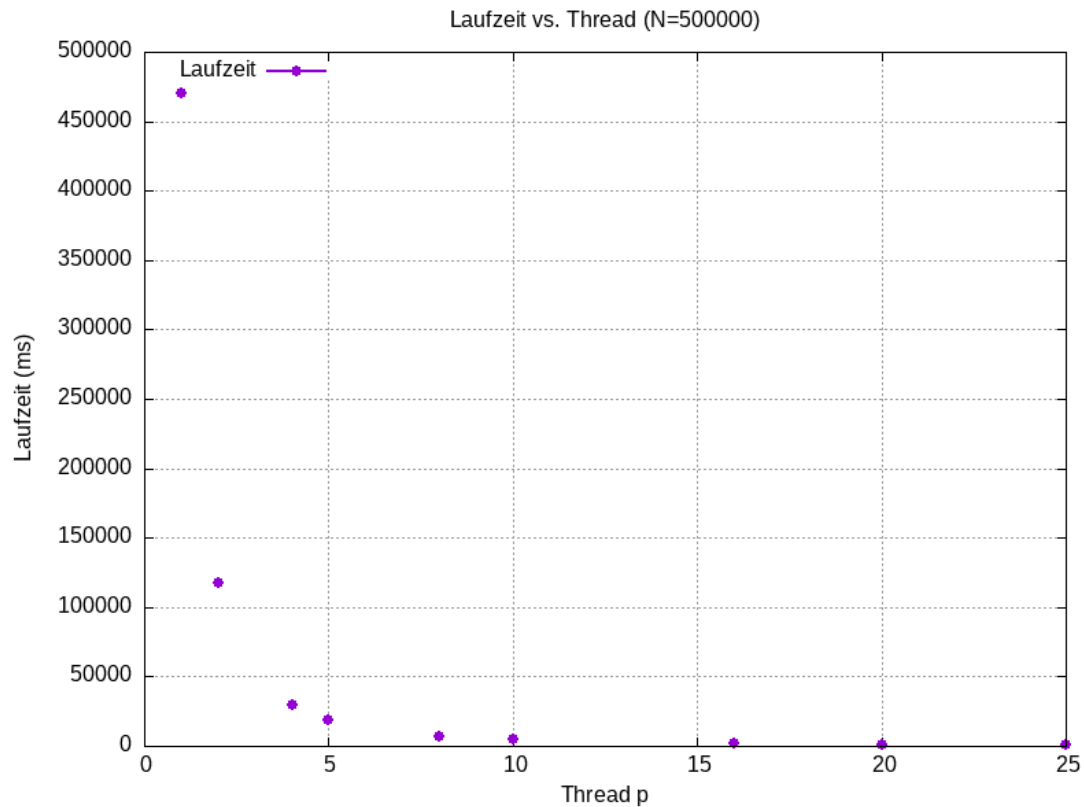


Abbildung 7: Laufzeit vs. Thread (N=500000)

Mit einem Kern ($p = 1$) dauert das Sortieren von $n = 500\,000$ Elementen etwa 470s. Bei zwei Kernen ($p = 2$) sinkt die Laufzeit auf ca. 118s, was einer knapp vierfachen Beschleunigung entspricht. Mit vier Kernen ($p = 4$) beträgt die Laufzeit etwa 30s, mit fünf Kernen ($p = 5$) rund 18s. Erhöht man die Kernzahl bis $p = 25$, so folgt die Laufzeit nahezu dem Skalierungsgesetz

$$T(p) = O(p^{-2}).$$

Verdoppelt man die Zahl der Kerne, reduziert sich die Laufzeit demnach um etwa den Faktor vier.

Grund hierfür ist, weil Bubble-Sort als lokale Sortierung eine Laufzeit von $(n/p)^2$ aufweist. Wenn man also die Anzahl der Kerne erhöht, verringert sich die Laufzeit quadratisch.

Für Bubble Sort ergibt sich die Gesamtlaufzeit

$$T(n, p) = O\left(\left(\frac{n}{p}\right)^2 + n\right).$$

Für $p < \sqrt{n}$ dominiert der quadratische Term $\left(\frac{n}{p}\right)^2$, und der Speedup folgt zu

$$r_n(p) = \frac{T(n, 1)}{T(n, p)} = \frac{O(n^2)}{O((n/p)^2)} = O(p^2).$$

Damit bestätigt sich das gemessene quadratische Abfallen der Laufzeit exakt durch die theoretische Vorhersage.

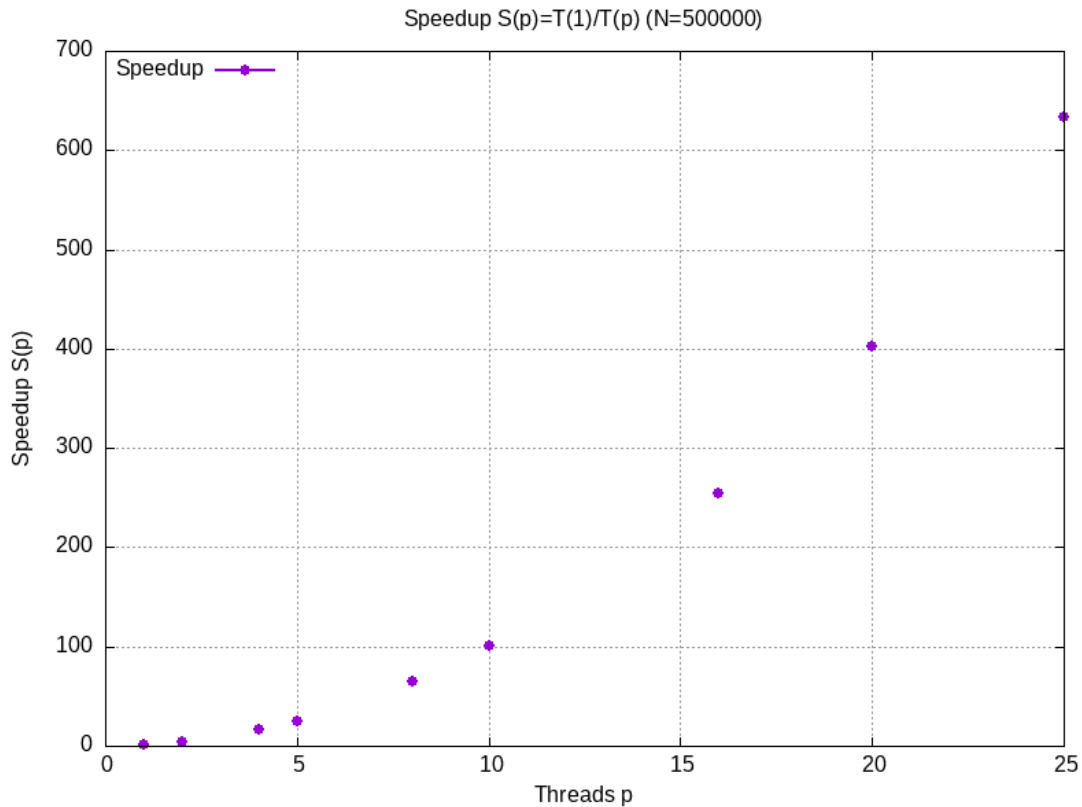


Abbildung 8: Speedup $S(p)=T(1)/T(p)$ ($N=500000$)

Die relative Beschleunigung startet bei

$$r_n(1) = 1, \quad r_n(2) \approx 4, \quad r_n(4) \approx 15, \quad r_n(8) = 64, \quad r_n(25) \approx 632.$$

Bis $p = 25$ zeigt $r_n(p)$ somit ein nahezu quadratisches Wachstum in p , d. h. $r_n(p) = \Theta(p^2)$.

Theoretische Einordnung (Aufgabe 3.3.5): Aus Aufgabe 3.3.5 wissen wir, dass für Bubble Sort

$$r_n(p) = O(p^2) \quad \text{für} \quad p < \sqrt{n}.$$

Da hier $\sqrt{500\,000} \approx 707$ und wir bis $p = 25$ messen, liegen wir eindeutig im $O(p^2)$ -Bereich. Die geringen Abweichungen resultieren aus dem noch vernachlässigbaren Merge-/Synchronisations-Overhead.

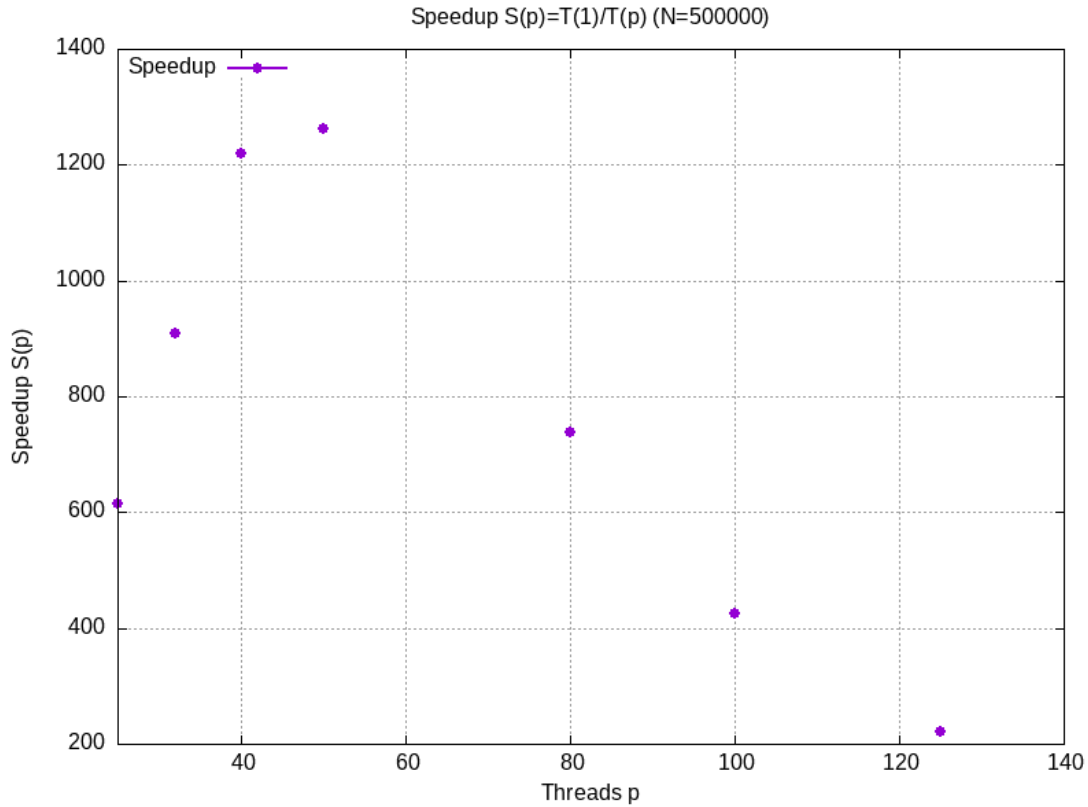


Abbildung 9: Speedup $S(p)=T(1)/T(p)$ ($N=500000$)

Hier kann man sehen, dass ab $p = 50$ der Zuwachs schon deutlich flacher wird. Bei $p = 128$ liegt die Beschleunigung nur noch bei ca. 220, anstelle des quadratischen Ideals. Grund hierfür ist, weil die Merge-Synchronisations-Phase $O(n)$ hierbei einfach teuer wird, weil die lokalen Blöcke so klein werden, dass das Zusammensetzen dominiert.

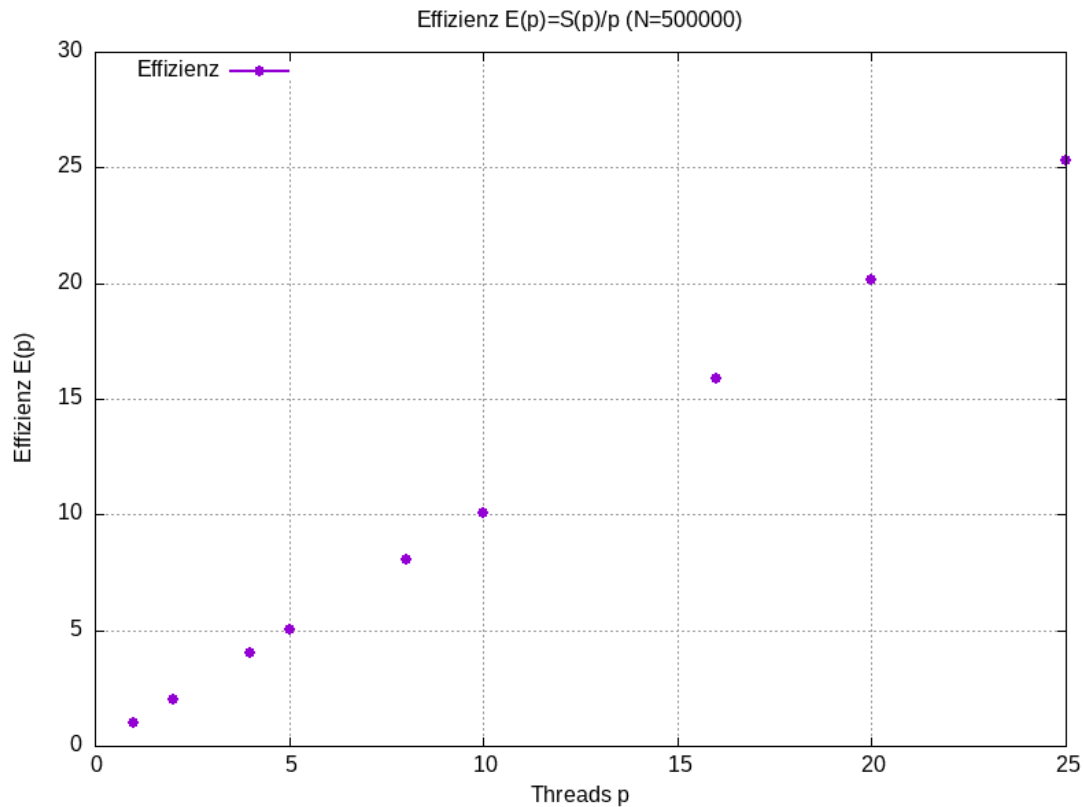


Abbildung 10: Effizienz $E(p)=S(p)/p$ ($N=500000$)

Gemessen ergibt sich für $n = 500\,000$:

$$e_n(1) = 1, \quad e_n(2) \approx 2, \quad e_n(4) \approx 5,8, \quad e_n(8) \approx 8, \quad e_n(25) \approx 25.$$

Bis $p = 25$ wächst die Effizienz nahezu linear mit p , denn die lokale Bubble-Sort-Arbeit dominiert so stark, dass durch Hinzufügen weiterer Kerne die Arbeit pro Kern sehr stark sinkt.

Theoretische Einordnung (Aufgabe 3.3.5) Aus Aufgabe 3.3.5 folgt

$$e_n(p) = \frac{r_n(p)}{p} = O(p) \quad \text{für } p < \sqrt{n}.$$

Da hier $\sqrt{500\,000} \approx 707$ und wir bis $p = 25$ messen, liegen wir eindeutig im $O(p)$ -Bereich. Die gemessene lineare Steigerung der Effizienz bestätigt damit die theoretische Vorhersage.

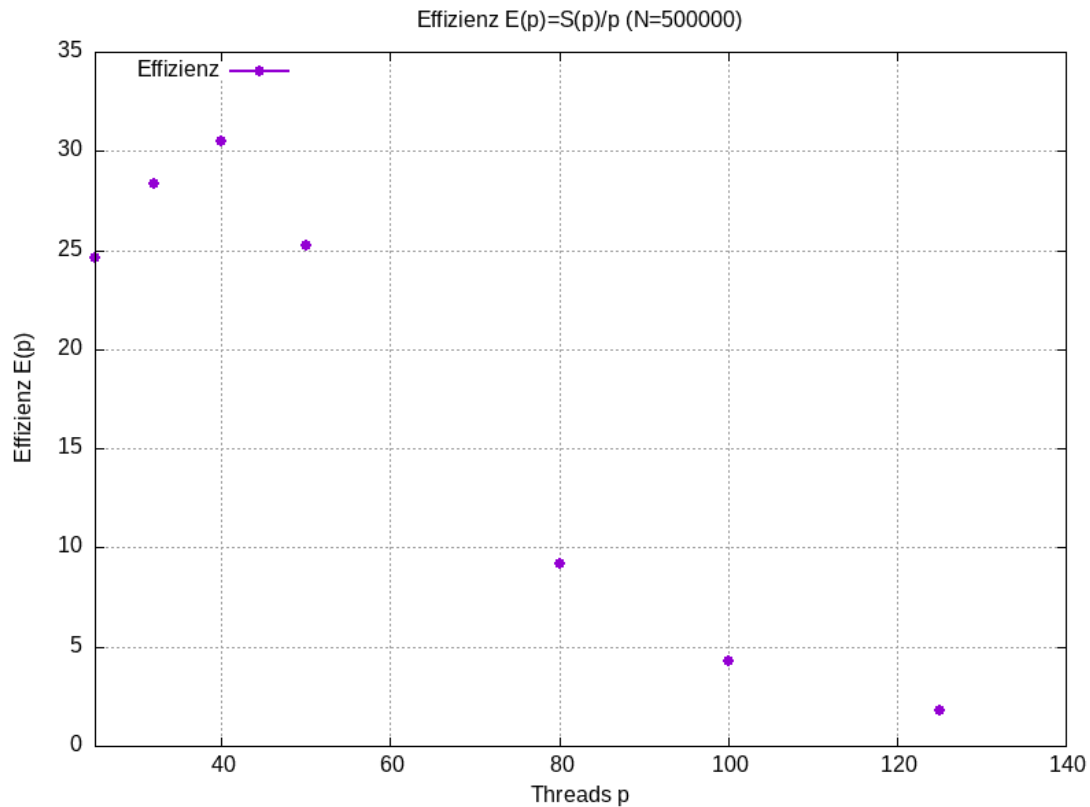


Abbildung 11: Effizienz $E(p)=S(p)/p$ ($N=500000$)

Sobald p deutlich größer wird, kippt die Effizienz. Sie fällt von rund 25 bei $p = 25$, auf unter 2, bei $p = 128$. Grund hierfür ist der, weil für sehr viele Kerne, der fest Merge-Overhead $O(n)$ nicht mehr zu vernachlässigen ist. Die Vorteile durch die Verkleinerung der lokalen Quadrate schrumpfen gegenüber dem Aufwand, alle Teilblöcke wieder zusammenzuführen.

Laufzeit, Beschleunigung und Effizienz als Tabelle für $N = 500\,000$ (Nachtrag)

Tabelle 3: Laufzeit, Speed-up und Effizienz für $N = 500\,000$ (Bubble Sort)

p	Laufzeit [ms]		Speed-up		Effizienz	
	Wert	%	Wert	%	Wert	%
1	470645	—	1.00	—	1.00	—
2	117949	−74.9	3.99	+299.0	1.995	+99.5
4	29263	−75.2	16.09	+303.0	4.02	+101.5
5	18646	−36.3	25.25	+57.0	5.05	+25.6
8	7294	−60.9	64.49	+155.0	8.06	+59.8
10	4662	−36.1	100.93	+56.5	10.09	+25.2
16	1850	−60.3	254.41	+152.0	15.90	+57.6
20	1169	−36.8	402.46	+58.1	20.12	+26.4
25	743	−36.4	633.65	+57.4	25.35	+26.0
32	518	−30.3	908.93	+43.5	28.40	+12.0
40	386	−25.5	1219.31	+34.2	30.48	+7.3
50	373	−3.4	1261.54	+3.4	25.23	−17.2
80	638	+71.0	737.69	−41.5	9.22	−63.5
100	1104	+73.0	426.33	−42.2	4.26	−53.8
125	2128	+92.8	221.19	−48.1	1.77	−58.5

Welche Variante des Algorithmus ist besser? Warum?

Wenn man den Vergleich für lokale Sortierung macht zwischen `Arrays.sort` mit einer Laufzeit-Äquivalenz von ca. Heapsort mit $O(n \log n)$, und Bubble-Sort mit $O((n/p)^2)$, dann ist in diesem Falle `Array.sort` klar im Vorteil.

Gründe hierzu sind zum Einen, dass es bei `Array.sort()` einen geringeren sequentiellen Aufwand gibt: `Array.sort()` braucht $O(\frac{n}{p} \log \frac{n}{p})$ pro Teilproblem. Bubble-Sort hingegen braucht $O((n/p)^2)$. Für typische n und p wächst $(n/p)^2$ deutlich höher als $(n/p) \log(n/p)$. Das kann man auch an den absoluten Laufzeiten sehen, wobei die Laufzeiten bei Bubble-Sort viel viel höher sind.

Des Weiteren bietet `Array.sort()` ein besseres Parallel-Scaling, denn: Bei `Array.sort()` bleibt man für $p \leq \log n$ nahe am idealen $1/p$ -Beschleunigung, wobei die Effizienz ca. 0,4–0,8 liegt, selbst bei 24 Kernen. Bubble-Sort kann zwar für den Anfang relativ hohe Beschleunigung liefern, bis $O(p^2)$, da die sequentielle Arbeit sehr dominant ist. Aber die absoluten Laufzeiten steigen hierbei dennoch enorm. Zusätzlich bricht die Effizienz spätestens ab mittleren Kernzahlen stark ab, weil das $O(n)$ -Merge dominiert.

Letztlich ist es auch so, dass bei großen n und guten p , selbst bei 128 Kernen, die Läufe von Bubble-Sort Minuten statt Sekunden dauern. Bei `Array.sort()` hingegen liefern dieselben n und p Laufzeiten im Sekundenbereich und bleiben innerhalb von einer vernünftigen Heap-Space-Grenze. Somit bietet Letzteres also auch praktische Zeiten und Speicher.

2.2 Bitonic Sort

Bitonic Sort ist ein paralleler Vergleichssortieralgorithmus, der insbesondere auf Architekturen mit Hypercube effizient implementiert werden kann. Er arbeitet auf Sequenzen der Länge $n = 2^k$ und besteht aus zwei Hauptschritten:

1. Bitonische Folge erzeugen Eine *bitonische Folge* $\{x_0, x_1, \dots, x_{n-1}\}$ ist definiert dadurch, dass sie zunächst (streng) monoton steigt und anschließend (streng) monoton fällt, oder dass sie durch zyklisches Schieben in eine solche Form überführt werden kann. Um eine beliebige Eingabefolge $A[0..n-1]$ bitonisch zu machen, sortiert man rekursiv die erste Hälfte $A[0..n/2-1]$ aufsteigend und die zweite Hälfte $A[n/2..n-1]$ absteigend. Die so erzeugte Gesamtliste ist bitonisch.

2. Bitonisches Mischen (Merge) Ist $B[0..n-1]$ bitonisch, lässt sie sich in zwei sortierte Teillisten zerlegen, indem man für alle $0 \leq i < n/2$

vergleicht und vertauscht $(B[i], B[i + n/2])$

so anwendet, dass jeweils das Minimum in Position i und das Maximum in Position $i + n/2$ verbleibt. Anschließend werden die beiden Hälften $B[0..n/2-1]$ und $B[n/2..n-1]$ jeweils rekursiv bitonisch gemischt. Am Ende dieser Phase liegen alle n Elemente in aufsteigender Reihenfolge vor.

2.2.1 Übungen zum Bitonic Sort

Verdeutlichen Sie sich die Arbeitsweise des parallelen Sortieralgorithmus "Bitonic Merging (BIM)"

Algorithm 2 BitonicSortMain

Require: Eingabegröße N , Threadanzahl P

Ensure: Global sortiertes Array über alle Prozessoren

```
1:  $m\_Array \leftarrow$  neues Array  $[0..P-1][0..N/P-1]$ 
2:  $barrier \leftarrow$  neue CyclicBarrier( $P$ )
3:  $processes \leftarrow$  neues Array  $[0..P-1]$ 
4: Fülle  $m\_Array$  mit Zufallswerten
5: for  $i = 0$  to  $P - 1$  do
6:    $processes[i] \leftarrow$  neue BitonicSortThread( $id = i, m\_Array[i], P, barrier$ )
7: end for
8: for  $i = 0$  to  $P - 1$  do
9:    $processes[i].start()$ 
10: end for
11: for  $i = 0$  to  $P - 1$  do
12:    $processes[i].join()$ 
13: end for
14: for  $i = 0$  to  $P - 1$  do
15:    $m\_Array[i] \leftarrow processes[i].getSortedSubArray()$ 
16: end for
```

Algorithm 3 BitonicSortThread

Require: Subarray local_i der Länge n/p , Prozess-ID i , Gesamtanzahl p , **barrier**

Ensure: local_i ist am Ende sortiert

```
1: procedure RUN
2:   PRESORT( $\text{local}_i$ )                 $\triangleright$  initiales Sortieren mit Arrays.sort()
3:   SORT(0,  $p$ , true)
4: end procedure
5: procedure SORT( $low$ ,  $count$ ,  $ascending$ )
6:   if  $count > 1$  then
7:     MAKEBITONIC( $low$ ,  $count$ ,  $ascending$ )
8:     BITONICMERGE( $low$ ,  $count$ ,  $ascending$ )
9:   end if
10: end procedure
11: procedure MAKEBITONIC( $low$ ,  $count$ ,  $ascending$ )
12:    $k \leftarrow count/2$ 
13:   if  $i < k$  then
14:     SORT( $low$ ,  $k$ , true)
15:   else
16:     SORT( $low + k$ ,  $k$ , false)
17:   end if
18: end procedure
19: procedure BITONICMERGE( $low$ ,  $count$ ,  $ascending$ )
20:   for  $dist \leftarrow count/2$  downto 1 by halving do
21:      $partner \leftarrow i \oplus dist$ 
22:     Sende  $\text{local}_i$  an  $\text{local}_{partner}$  und empfange other
23:      $\text{local}_i \leftarrow \text{MERGEANDSWAP}(\text{local}_i, \text{other}, ascending)$ 
24:   end for
25: end procedure
26: procedure GETSORTEDSUBARRAY
27:   return  $\text{local}_i$ 
28: end procedure
```

Bestimmen Sie die Laufzeit des BIM-Algorithmus und vergleichen diese mit dem Odd Even Merge Splitting

Bitonic Sort auf p Prozessoren zerfällt in zwei Phasen:

1. Lokales Sortieren Jeder der p Kerne hält ein Teilarray der Länge

$$m = \frac{n}{p}$$

und sortiert dieses mit einem sequentiellen Vergleichssortierverfahren (z. B. Java's `Arrays.sort()`). Da ein solcher Algorithmus $m \log m$ -Schritte benötigt, ergibt sich pro Kern

$$O(m \log m) = O\left(\frac{n}{p} \log\left(\frac{n}{p}\right)\right).$$

Weil alle Kerne diese Arbeit gleichzeitig leisten, ist dies der dominierende Beitrag der ersten Phase.

2. Bitonisches Zusammenführen (Merge) Die bereits sortierten Blöcke müssen nun zu einer global sortierten Sequenz verschmolzen werden. Bitonic Sort verwendet dafür ein Merge-Netzwerk mit $\log_2 p$ Stufen. In der k -ten Stufe gilt

$$\text{dist} = \frac{p}{2^k}, \quad k = 1, 2, \dots, \log_2 p.$$

Jeder Kern i führt dabei pro Runde folgende Schritte aus:

1. **Referenztausch ($O(1)$):** Der Zeiger auf das lokale Array `locali` wird mit dem Partner $i \oplus \text{dist}$ ausgetauscht.
2. **mergeAndSwap ($O(m)$):** Die beiden sortierten Arrays der Länge m werden elementweise in ein Hilfsarray gemischt und per Dreiecksswap zurückkopiert. Dieser Vorgang benötigt $O(m)$ Schritte.

Da jede der $\log_2 p$ Runden auf jedem Kern insgesamt $O(1) + O(m) = O(m)$ Arbeit verursacht, summiert sich die Merge-Phase zu

$$O(m) \times \log_2 p = O\left(\frac{n}{p} \log p\right).$$

Gesamtlaufzeit Die Gesamtzeit ist die Summe beider Phasen:

$$T_{\text{bitonic}}(n, p) = O\left(\frac{n}{p} \log \frac{n}{p}\right) + O\left(\frac{n}{p} \log p\right) = O\left(\frac{n}{p} [\log \frac{n}{p} + \log p]\right) = O\left(\frac{n}{p} \log n\right),$$

wobei wir im letzten Schritt nutzen, dass $\log(n/p) + \log p = \log n$.

Damit erzielt Bitonic Sort nahezu linearen Speed-up in p , nur abgeschwächt um den Faktor $\log n$.

Beschleunigung

Die *relative Beschleunigung* $r_n(p)$ gibt an, um welchen Faktor Bitonic Sort auf p Kernen schneller läuft als auf einem einzelnen Kern. Sie ist definiert als

$$r_n(p) = \frac{T(n, 1)}{T(n, p)}.$$

1. Einzelkern-Laufzeit Auf einem Kern sortiert man das gesamte Array der Länge n in

$$T(n, 1) = O(n \log n).$$

2. Mehrkern-Laufzeit Auf p Kernen hat Bitonic Sort die Laufzeit

$$T(n, p) = O\left(\frac{n}{p} \log n\right),$$

weil sich die Arbeit $n \log n$ in etwa gleichmäßig auf die p Kerne verteilt.

3. Speedup Setzt man diese Größen in die Definition ein, kürzen sich n und $\log n$ heraus:

$$r_n(p) = \frac{O(n \log n)}{O\left(\frac{n}{p} \log n\right)} = O\left(\frac{n \log n}{(n/p) \log n}\right) = O(p).$$

Daraus folgt, dass Bitonic Sort nahezu *linear* in p skaliert: Verdoppelt man die Kernzahl, verdoppelt sich annähernd auch der Speedup.

Effizienz

Die *Effizienz* $e_n(p)$ misst, wie viel der theoretisch möglichen linearen Beschleunigung pro Kern tatsächlich erreicht wird:

$$e_n(p) = \frac{r_n(p)}{p}.$$

Mit $r_n(p) = O(p)$ ergibt sich

$$e_n(p) = \frac{O(p)}{p} = O(1).$$

Intuitive Erklärung

- **Idealfall:** Ohne jeglichen Overhead würde jeder Kern exakt seinen Anteil an Arbeit leisten, der Speedup wäre $r_n(p) = p$ und damit $e_n(p) = 1$.
- **Praktische Overheads:** Bitonic Sort führt $\log p$ Merge-Runden durch, was einen kleinen zusätzlichen Aufwand erzeugt. Da dieser Overhead aber nur einen weiteren Poly-Log-Faktor bringt, bleibt $e_n(p)$ asymptotisch konstant.
- **Ergebnis:** Bitonic Sort erreicht eine *konstante Effizienz* $e_n(p) = \Theta(1)$. In Messungen zeigt sich typischerweise $e_n(p) \approx 0,8\text{--}0,9$ für moderate Kernzahlen, ohne mit wachsendem p deutlich abzufallen.

Vergleich mit Odd Even Merge-Splitting

Wir fassen zunächst die theoretischen Laufzeiten noch einmal zusammen:

- **Odd-Even Merge-Splitting (OEMS):**

$$T_{\text{OEMS}}(n, p) = O\left(\frac{n}{p} \log \frac{n}{p}\right) + O(n) = O(\max\{\frac{n}{p} \log \frac{n}{p}, n\}).$$

- Für $p \leq \log n$ ist $T_{\text{OEMS}}(n, p) = O(\frac{n}{p} \log n)$.
- Für $p > \log n$ dominiert der $O(n)$ -Term und $T_{\text{OEMS}}(n, p) = O(n)$.

- **Bitonic Sort:**

$$T_{\text{bitonic}}(n, p) = O\left(\frac{n}{p} \log \frac{n}{p}\right) + O\left(\frac{n}{p} \log p\right) = O\left(\frac{n}{p} \log n\right).$$

- Für alle $1 \leq p \leq n$ fällt $T_{\text{bitonic}}(n, p)$ weiterhin mit $1/p$, nur gebremst durch $\log n$.

Speedup (Beschleunigung)

$$r_{\text{OEMS}}(p) = \begin{cases} O(p), & p \leq \log n, \\ O(\log n), & p > \log n, \end{cases} \quad r_{\text{bitonic}}(p) = O(p) \quad \forall p.$$

Fazit: OEMS skaliert nur linear bis $p \approx \log n$ und flacht dann ab, Bitonic bleibt linear in p .

Effizienz

$$e_{\text{OEMS}}(p) = \begin{cases} O(1), & p \leq \log n, \\ O(\frac{\log n}{p}), & p > \log n, \end{cases} \quad e_{\text{bitonic}}(p) = O(1) \quad \forall p.$$

Fazit: OEMS verliert ab $p > \log n$ Effizienz, während Bitonic Sort seine konstante Auslastung aller Kerne beibehält.

Schlussfolgerung Bitonic Sort bietet im asymptotischen Vergleich eine bessere Skalierung, insbesondere für viele Prozessoren ($p > \log n$):

- *OEMS* bleibt wegen des festen Merge-Overheads von $O(n)$ bei höheren Kernzahlen auf einem konstanten Laufzeitsniveau und kann keine weitere Beschleunigung erzielen.
- *Bitonic Sort* nutzt jeden zusätzlichen Kern weiterhin aus (bis hin zu $p = O(n)$), da die Gesamtarbeit immer noch proportional zu n/p fällt.

2.2.2 Assignment 2

In Assignment 2 wird der Bitonic Merge-Splitting (BIM)–Algorithmus untersucht. Dazu sind folgende Schritte durchzuführen:

1. **Implementierung:** Der sequentielle Sortierschritt verwendet Java's `Arrays.sort()`. Die BIM-Logik wird in einer Klasse `BitonicSortMain` (Treibercode) und einer Klasse `BitonicSortThread` (Parallelcode) umgesetzt.
2. **Laufzeitmessungen:**
 - Ein- und dieselbe Zufalls-Eingabe (mit gegebener Größe N) wird mehrfach (dreimal) sortiert; jeweils wird die schnellste Laufzeit verwendet.
 - Die Anzahl der Threads/Kerne p wird variiert — in diesem Versuch für $p = 1, 2, 4, 8, \dots, 1024$ (Potenzen von zwei).
 - Gemessen wird für zwei Problemgrößen

$$N_1 = 921\,600\,000, \quad N_2 = 1\,843\,200\,000.$$

3. **Auswertung:**

- Bestimmung der *relativen Beschleunigung* $r_n(p) = T(n, 1)/T(n, p)$ und der *Effizienz* $e_n(p) = r_n(p)/p$.

- Grafische Darstellung von Laufzeit, Speed-up und Effizienz in Abhängigkeit von p .
 - Vergleich der gemessenen Kurven mit den theoretischen Vorhersagen aus Abschnitt 4.3.2 (Bitonic Sort) und Abschnitt 3.3.2 (Odd–Even Merge Splitting).
4. **Validierung:** Ein automatischer Check stellt sicher, dass nach Beendigung alle n Eingangselemente korrekt sortiert und vollständig vorhanden sind.

Problemgröße $n = 921\,600\,000$

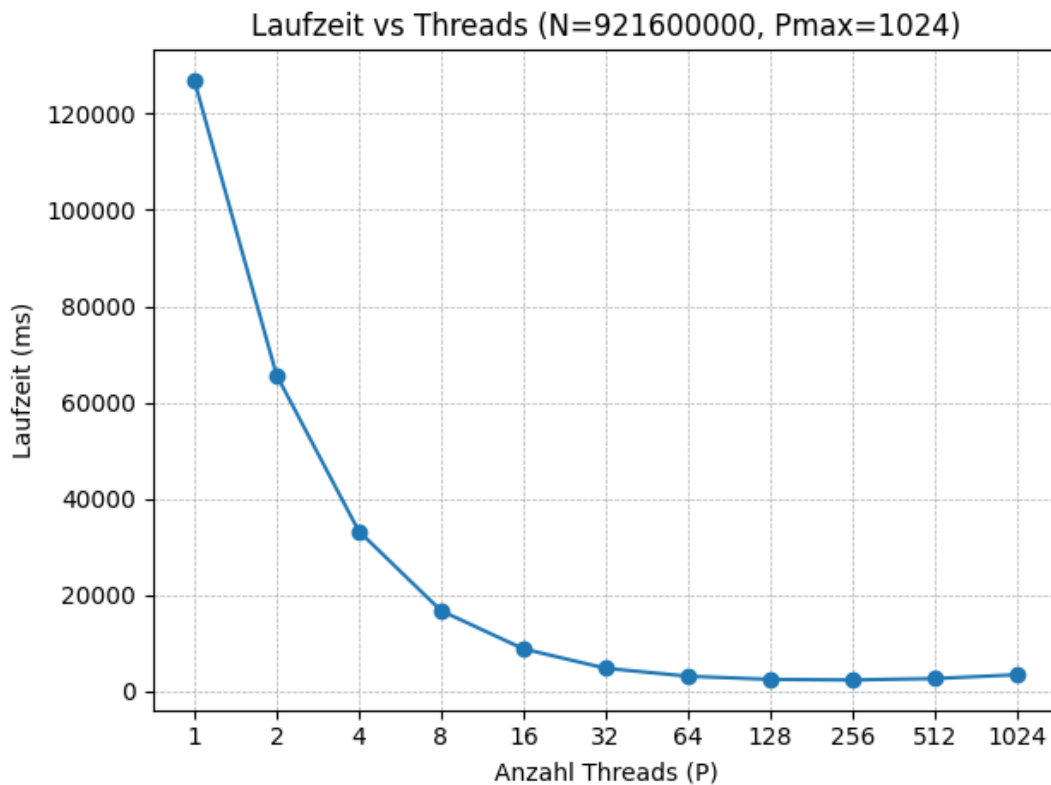


Abbildung 12: Laufzeit vs. Threads (N=921600000, Pmax=1024)

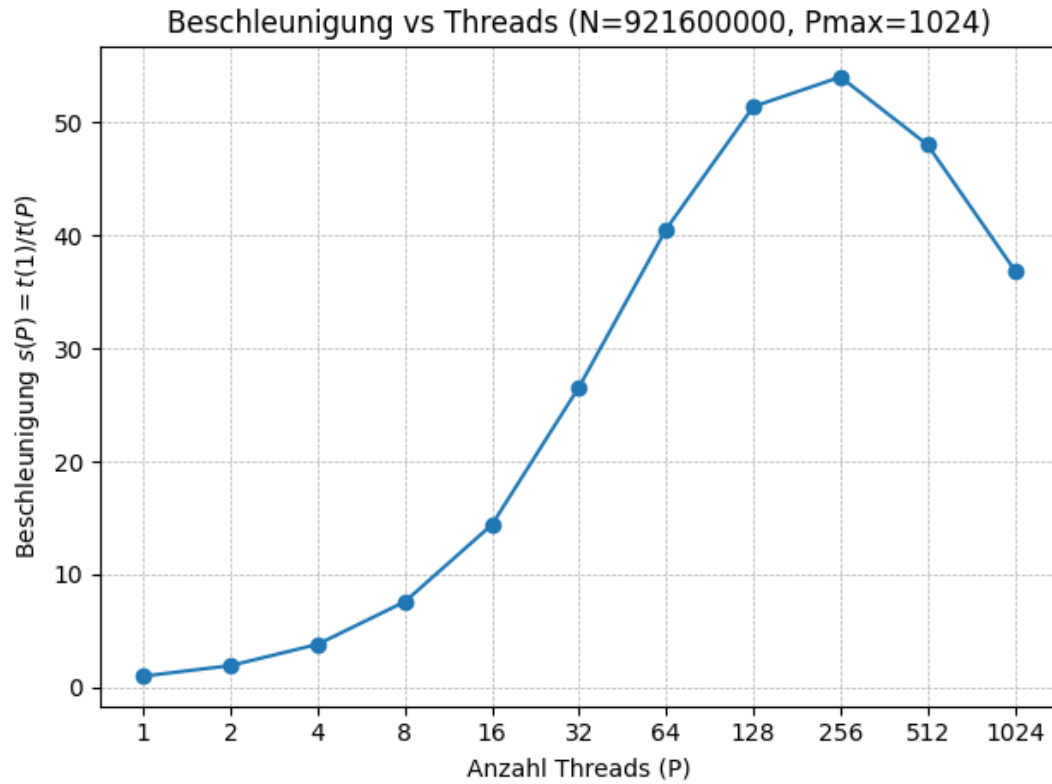


Abbildung 13: Beschleunigung vs. Threads (N=921600000, Pmax=1024)

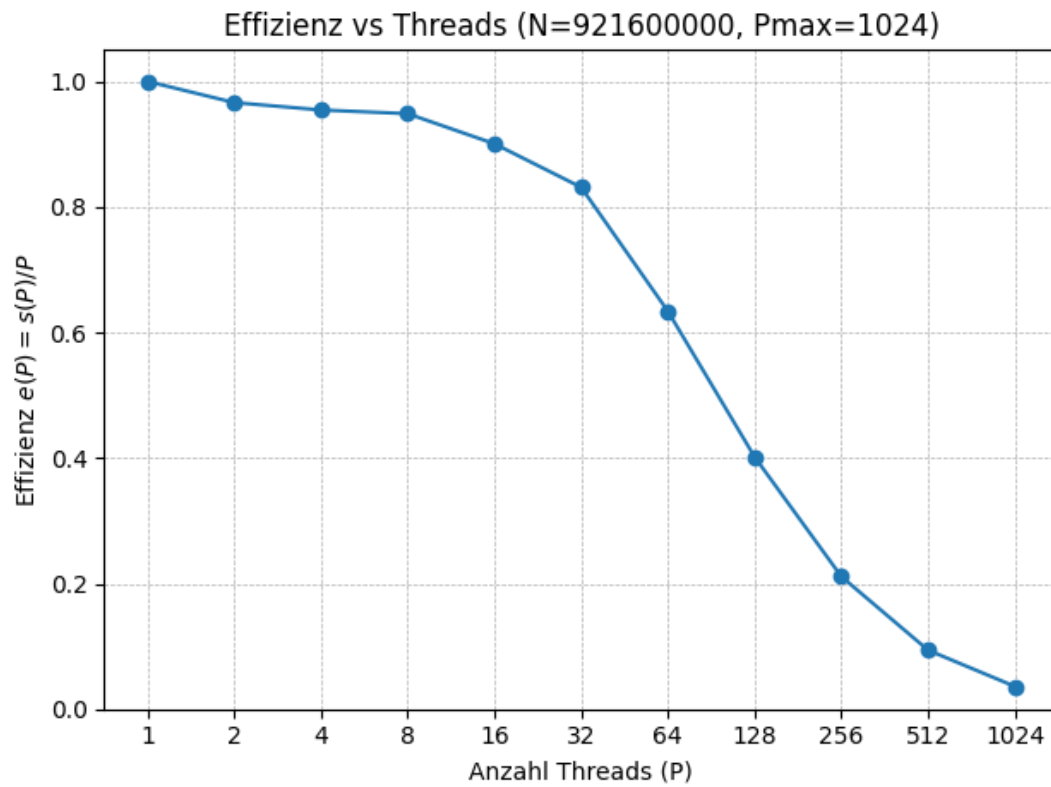


Abbildung 14: Effizienz vs. Threads (N=921600000, Pmax=1024)

Tabelle 4: Laufzeit, Speed-up und Effizienz für Bitonic Merge (gemessen vs. theoretisch)

p	$T(p)$ [ms]	ΔT [%]	$S(p)$	ΔS [%]	$e(p)$	Δe [%]	S_{theo}	e_{theo}
1	126967	—	1.00	—	1.00	—	1.00	1.00
2	65699	48.2	1.93	93.0	0.97	3.0	2.00	1.00
4	33256	49.4	3.82	98.2	0.95	2.1	4.00	1.00
8	16731	49.7	7.59	98.6	0.95	0.0	8.00	1.00
16	8811	47.3	14.41	90.0	0.90	5.3	16.00	1.00
32	4773	45.8	26.60	84.5	0.83	7.8	32.00	1.00
64	3130	34.4	40.56	52.5	0.63	24.1	64.00	1.00
128	2470	21.0	51.40	26.7	0.40	36.5	128.0	1.00
256	2350	4.9	54.01	5.1	0.21	47.5	256.0	1.00
512	2644	—	12.5	48.01	0.09	57.1	512.0	1.00
1024	3439	30.1	36.90	23.1	0.04	55.6	1024	1.00

Vergleich Messung vs. Theorie In der Tabelle sieht man für jede Kernzahl p zunächst die gemessene Laufzeit und direkt daneben, um wie viel Prozent sie gegenüber der vorherigen Konfiguration ab- oder angestiegen ist. Danach folgt die errechnete Beschleunigung mit seiner prozentualen Änderung, sowie die Effizienz samt deren Änderung. In den letzten beiden Spalten stehen die theoretisch idealen Werte: eine lineare Beschleunigung und eine konstante Effizienz.

Wesentlich ist: Bis etwa $p = 16$ wächst die gemessene Beschleunigung fast linear und die Effizienz bleibt hoch (nahe 0,9 - 1,0), also sehr nahe am Optimum. Ab $p = 32$ jedoch verlangsamt sich der Zugewinn deutlich - der prozentuale Differenz von Laufzeit und Beschleunigung nimmt ab, und die Differenz fällt auf die Werte unter 0,7 ab. Das zeigt klar, dass der Kommunikations- und Merge-Overhead bei höheren Prozessorzahlen die Skalierung bremst. Die theoretischen Werte dienen hier als Referenz: Während Bitonic Merge in der Praxis bei kleinen bis mittleren p nahezu ideal skaliert, kann es ab einer gewissen Kernzahl nicht mehr die volle lineare Beschleunigung und konstante Auslastung erreichen.

Vergleich BIM vs. OEM Um einen „Äpfel-mit-Äpfel“-Vergleich zu ermöglichen, werden Bitonic Merge und Odd-Even Merge-Splitting bei

- gleicher Problemgröße $N = 921\,600\,000$,
- identischer Prozessorzahl $p \in \{1, 2, 4, 8, 16\}$,

- und demselben lokalen Sortierschritt (`Arrays.sort()`)

nebeneinander betrachtet.

2^*p	T_{bitonic}		S_{bitonic}		e_{bitonic}		T_{OEMS}		S_{OEMS}		e_{OEMS}	
	Wert	%	Wert	%	Wert	%	Wert	%	Wert	%	Wert	%
1	126967	—	1.00	—	1.00	—	127082	—	1.00	—	1.00	—
2	65699	-48.2	1.93	+93	0.97	-3	66627	-47.6	1.91	+91	0.96	-4
4	33256	-49.4	3.82	+98	0.96	-1	37471	-43.8	3.39	+77	0.85	-11
8	16731	-49.7	7.59	+99	0.95	-1	22752	-39.3	5.59	+65	0.70	-18
16	8811	-47.3	14.40	+90	0.90	-5	15266	-32.9	8.32	+49	0.52	-26

In der Tabelle stehen für jede Prozessorzahl p die gemessene Laufzeit T , der daraus berechnete Speed-up $S = T(1)/T(p)$ und die Effizienz $e = S/p$. Neben jedem Wert ist jeweils die prozentuale Veränderung gegenüber der vorhergehenden Zeile angegeben. Der linke Block zeigt die Kennzahlen für Bitonic Merge, der rechte für Odd–Even Merge–Splitting (OEMS).

Man erkennt, dass bei kleinen p beide Algorithmen vergleichbare Speed-ups liefern, Bitonic Merge aber ab $p = 4$ zunehmend besser skaliert. Die Effizienz von OEMS sinkt schon ab $p = 4$ deutlich, während Bitonic Merge bis $p = 16$ über 0,9 bleibt. Das deutet darauf hin, dass Bitonic Merge den parallelen Merge-Overhead besser auf die Kerne verteilt, während OEMS—bedingt durch seinen festen $O(n)$ -Merge-Anteil—ab einer gewissen Kernzahl nicht mehr sinnvoll weiter beschleunigt.

Problemgröße $n = 1\,843\,200\,000$

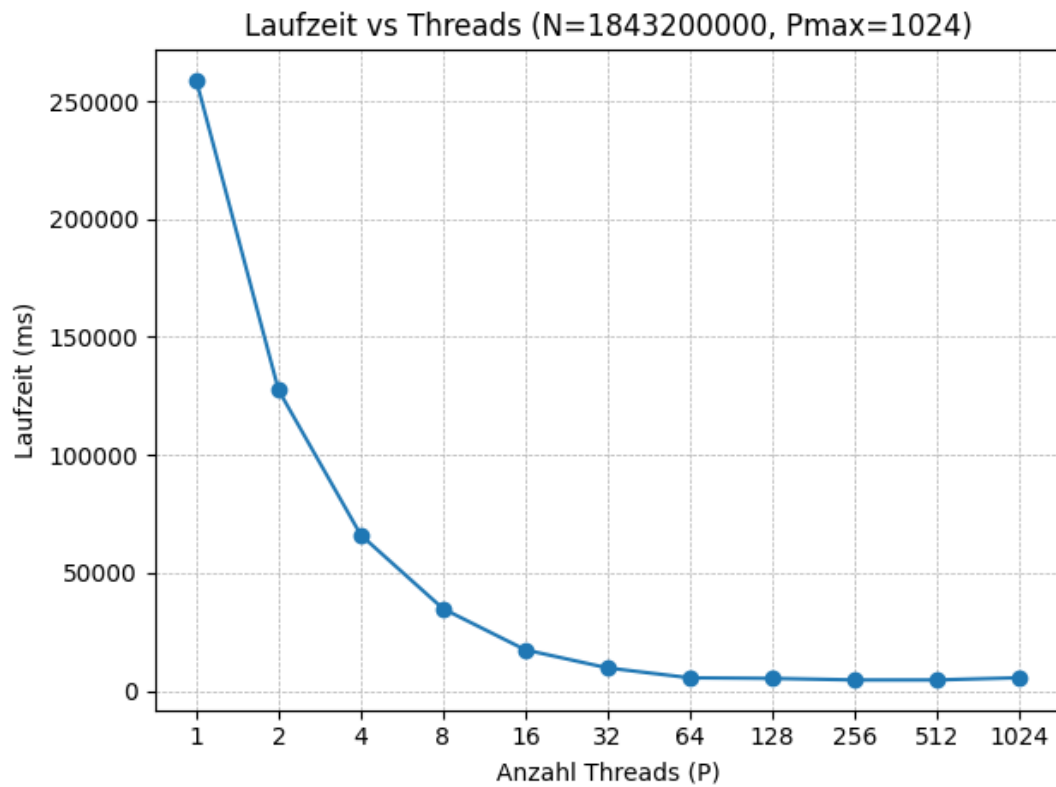


Abbildung 15: Laufzeit vs. Threads (N=1843200000, Pmax=1024)

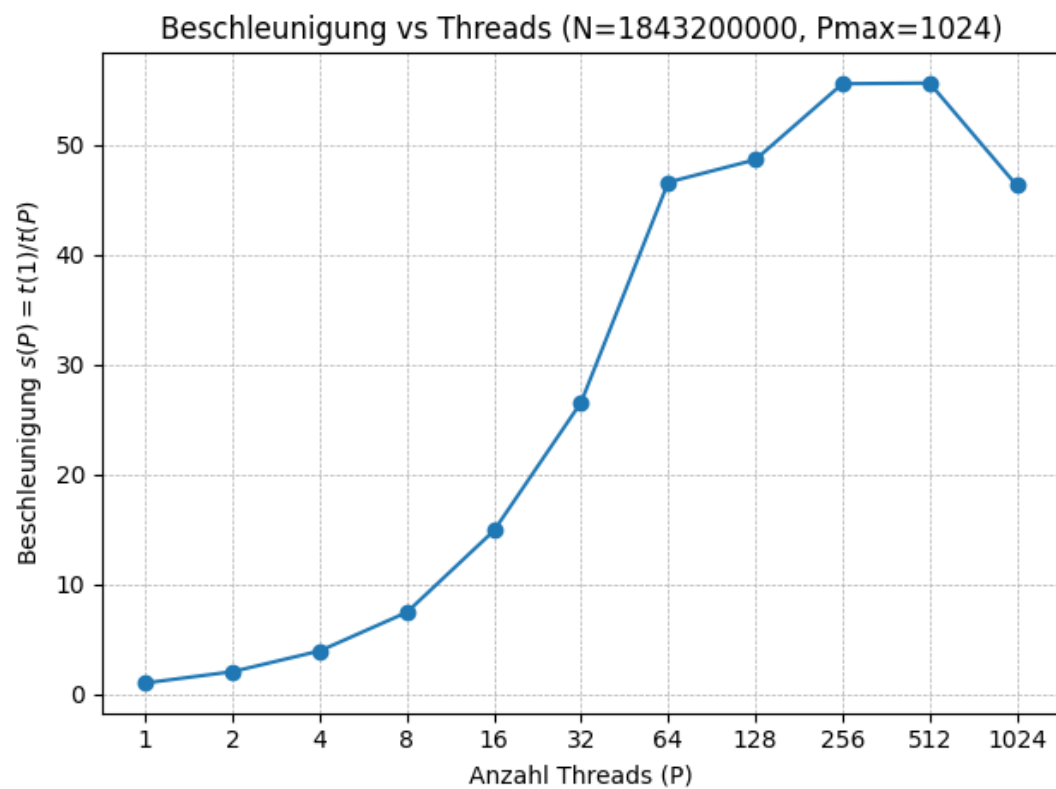


Abbildung 16: Beschleunigung vs. Threads (N=1843200000, Pmax=1024)

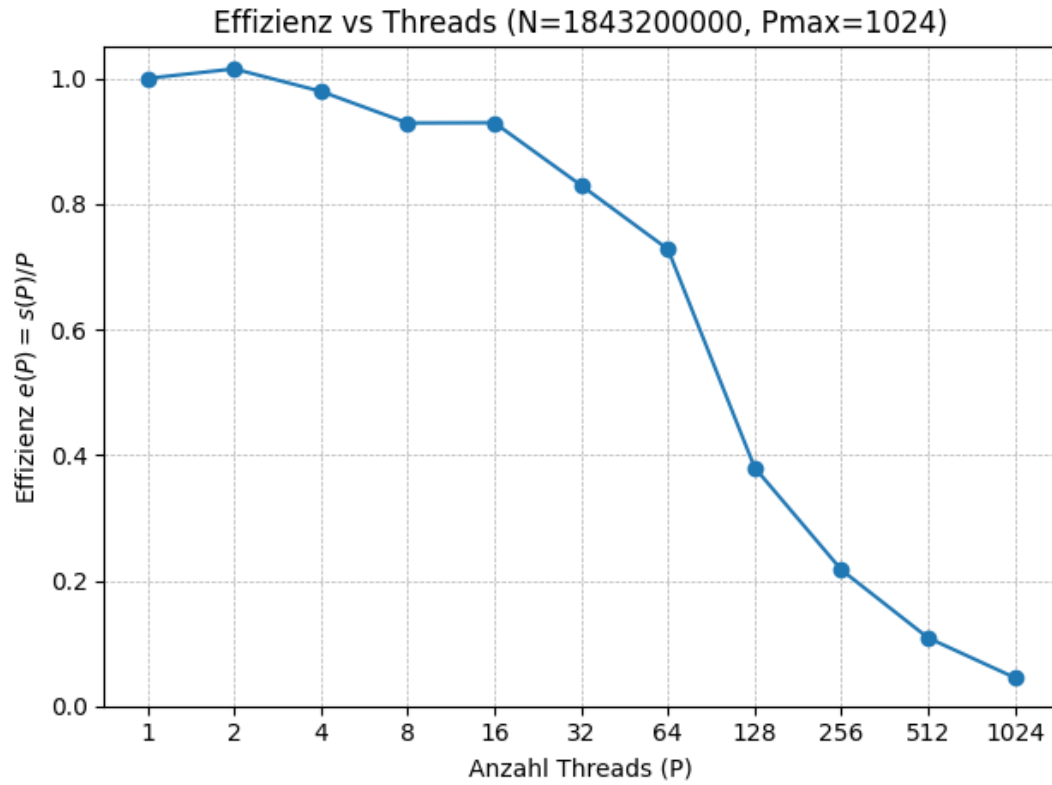


Abbildung 17: Effizienz vs. Threads (N=1843200000, Pmax=1024)

Tabelle 5: Laufzeit, Speed-up und Effizienz von Bitonic Merge (gemessen vs. theoretisch) für $N = 1\,843\,200\,000$

2^*p	$T(p)$ [ms]		$S(p)$		$e(p)$		S_{theo}		e_{theo}	
	Wert	%	Wert	%	Wert	%	Wert	—	Wert	—
1	258907	—	1.00	—	1.00	—	1.00		1.00	
2	127522	-50.8	2.03	+103.0	1.015	+1.5	2.00		1.00	
4	66088	-48.2	3.92	+93.6	0.980	-3.4	4.00		1.00	
8	34840	-47.3	7.43	+89.5	0.929	-5.1	8.00		1.00	
16	17412	-50.0	14.87	+100.1	0.930	0.1	16.00		1.00	
32	9753	-44.0	26.54	+78.5	0.829	-10.8	32.00		1.00	
64	5558	-43.0	46.56	+75.4	0.727	-12.3	64.00		1.00	
128	5324	-4.2	48.62	+4.4	0.379	-47.9	128.0		1.00	
256	4661	-12.4	55.56	+14.3	0.217	-42.1	256.0		1.00	
512	4656	-0.1	55.61	+0.1	0.109	-49.8	512.0		1.00	
1024	5593	+20.1	46.28	-16.8	0.045	-58.8	1024.		1.00	

Vergleich Messung vs. Theorie Man erkennt, dass Bitonic Merge bis $p = 16$ sehr gut skaliert: Laufzeit halbiert sich annähernd bei jeder Verdopplung von p , Speed-up wächst fast linear und Effizienz bleibt nahe 1. Ab $p = 32$ bricht die Effizienz jedoch um über 10 % pro Schritt ein und die Laufzeitvorteile flachen

ab – ein Zeichen dafür, dass der nicht-parallele Merge-Overhead und die Synchronisation bei vielen Kernen ins Gewicht fallen. Damit bestätigt sich auch für die größere Problemgröße die theoretische Vorhersage $T(n, p) = O(\frac{n}{p} \log n)$ nur bis zu einer mittleren Kernzahl, während danach konstante Overheads dominieren.

Vergleich BIM vs. OEM

Tabelle 6: Laufzeit, Speed-up und Effizienz von Bitonic Merge (BIM) vs. Odd-Even Merge-Splitting (OEMS)

2^*p	T_{BIM} [ms]		S_{BIM}		e_{BIM}		T_{OEMS} [ms]		S_{OEMS}		e_{OEMS}	
	Wert	%	Wert	%	Wert	%	Wert	%	Wert	%	Wert	%
1	258907	—	1.00	—	1.00	—	258615	—	1.00	—	1.00	—
2	127522	-50.7	2.03	+103.0	1.02	+1.5	135966	-47.4	1.90	+90.2	0.95	-4.9
4	66088	-48.2	3.92	+93.0	0.98	-3.5	81071	-40.4	3.19	+67.7	0.80	-16.1
8	34840	-47.3	7.43	+89.7	0.93	-5.2	55927	-31.0	4.62	+45.0	0.58	-27.5
16	17412	-50.0	14.87	+100.1	0.93	+0.0	45582	-18.5	5.67	+22.7	0.35	-38.7

Bis $p = 4$ liefern beide Algorithmen sehr ähnliche Speed-ups. Ab $p = 8$ skaliert BIM deutlich besser: Die Laufzeit reduziert sich hier bei jeder Verdopplung um rund 47 % (OEMS nur um 31 %), der Speed-up wächst um fast 90 % (OEMS um 45 %) und die Effizienz fällt bei BIM nur um etwa 5 % ab, während sie bei OEMS um über 25 % einbricht. Damit steht fest, dass Bitonic Merge den Merge-Overhead auf viele Kerne effektiver verteilt und auch bei höheren Kernzahlen eine hohe Auslastung beibehält, während OEMS ab mittleren p nicht mehr nachhaltig beschleunigt.

2.3 Merge Sort

2.3.1 Übungen zum parallelen Merge Sort

2.3.2 Assignment 3

2.4 Amdahls Gesetz und Gustafsons Interpretation

Amdahls Gesetz Amdahls Gesetz beschreibt die theoretische Grenze der Beschleunigung eines Algorithmus durch Parallelisierung. Es geht davon aus, dass ein Teil eines Algorithmus (der sequentielle Anteil) sich nicht parallelisieren lässt. Dadurch ergibt sich, dass selbst bei einer großen Anzahl von Prozessoren

die maximal erreichbare Beschleunigung begrenzt ist. Amdahl formuliert die Beschleunigung $b(N)$ bei N Prozessoren als:

$$b(N) = \frac{1}{s + \frac{p}{N}}$$

wobei s der sequentielle und p der parallelisierbare Anteil mit $s + p = 1$ ist. Das Gesetz macht deutlich, dass eine Reduktion des sequentiellen Anteils entscheidend ist, um hohe Skalierbarkeit zu erreichen.

Gustafsons Interpretation Im Gegensatz dazu geht Gustafson davon aus, dass bei steigender Prozessoranzahl auch die Problemgröße wächst. Somit bleibt die Laufzeit des parallelen Algorithmus konstant, aber es kann mehr Arbeit in gleicher Zeit verrichtet werden. Die Beschleunigung steigt daher linear mit der Prozessoranzahl:

$$b(N) = s + p \cdot N$$

Dies ergibt ein positiveres Bild der Parallelisierung im praktischen Einsatz und zeigt, dass mit wachsendem Problemumfang höhere Beschleunigungen erreichbar sind – solange der parallelisierbare Anteil groß genug ist.

3 Schluss

3.1 Reflexion

3.2 Fazit